
Bases de Datos

Clave-Valor
Redis

¿Qué es Redis?

Redis es una base de datos NoSQL de tipo clave-valor, que se ejecuta en memoria (RAM) y se caracteriza por ser muy rápida y flexible.

Su nombre viene de REmote DIctionary Server

Características principales:

- **En memoria:** los datos se guardan principalmente en RAM, lo que permite tiempos de respuesta menores a 1 milisegundo.
- **Modelo clave-valor:** cada dato se accede mediante una clave única, como si fuera un diccionario.
- **Tipos de datos avanzados:** no solo cadenas (strings), también listas, conjuntos, hashes, conjuntos ordenados (zsets), contadores, streams, etc.
- **Persistencia opcional:** aunque trabaja en memoria, puede guardar datos en disco con dos mecanismos:
 - RDB (snapshots): guarda el estado de la base cada cierto tiempo.
 - AOF (append only file): guarda todas las operaciones para reconstruir el estado.
- **Alto rendimiento:** muy usado en escenarios de tiempo real (chat, juegos, estadísticas).
- **Compatible con muchos lenguajes:** Python, Java, C#, Node.js, PHP, etc.

¿Para qué se usa Redis?

- Caché de datos (ej: resultados de consultas frecuentes).
- Gestión de sesiones de usuarios en aplicaciones web.
- Contadores y estadísticas en tiempo real.
- Rankings y sistemas de puntuación.
- Chat y mensajería.
- Carritos de compra en e-commerce.

Redis vs. Mysql

Característica	Redis	MySQL (Relacional)
Modelo de datos	Clave-Valor (NoSQL). Soporta Strings, Listas, Sets, Hashes, ZSets, Streams.	Tablas con filas y columnas (modelo relacional).
Velocidad	Muy rápida (opera en memoria RAM). Respuestas < 1 ms.	Más lenta porque accede a disco, aunque con índices puede ser eficiente.
Persistencia	Opcional. Puede trabajar solo en memoria o guardar datos en disco con RDB/AOF.	Siempre persiste en disco.
Escalabilidad	Muy buena para operaciones distribuidas, replicación y clustering.	Escalabilidad más limitada; requiere particiones o réplicas manuales.
Consultas	Comandos simples sobre claves (GET, SET, SADD, HGETALL, etc.). No tiene JOINS.	Lenguaje SQL: SELECT, JOIN, GROUP BY, etc. Consultas complejas y estructuradas.
Casos de uso	Caché, sesiones de usuario, colas de mensajes, rankings, estadísticas en tiempo real.	Gestión de datos estructurados y consistentes: sistemas bancarios, inventarios, aplicaciones empresariales.
Consistencia (ACID)	No es completamente ACID (aunque soporta transacciones simples). Prioriza rendimiento y disponibilidad.	Totalmente ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).

Redis vs. Mysql

```
CREATE TABLE usuarios (  
  id INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  edad INT  
);  
INSERT INTO usuarios VALUES (1, 'Ana', 25);  
SELECT * FROM usuarios WHERE id = 1;
```

MySQL

```
HSET usuario:1 nombre "Ana" edad 25  
HGETALL usuario:1
```

Redis

En redis todo se accede a partir de la clave, mientras que en MySQL utilizamos consultas estructuradas

Instalacion

- `sudo apt-get update`
- `sudo apt-get install redis-server php-redis`
- Ejecutar el cliente: **redis-cli**, debería aparecer algo así:
 - `127.0.0.1:6379>`
 - `127.0.0.1:6379>ping`
 - `127.0.0.1:6379> PONG`

Linux

github.com/MSOpenTech/redis/releases

- Descargue el archivo .msi o .zip, este tutorial le permitirá descargar el último archivo zip.
- Extraiga el archivo zip en un directorio.
- Ejecutar `redis-server.exe`.
- Ejecutar `redis-cli.exe`, después de ejecutar con éxito el servidor de redis.
- El comando PING se usa para probar si una conexión está aún activa

Windows

`docker run -d --name my-redis -p 6379:6379 redis:latest`

Docker

Strings

Es una secuencia de bytes: puede ser texto, números, o incluso binarios (como imágenes codificadas o JSON).

Cada clave puede almacenar un string de hasta 512 MB.

Son atómicos: las operaciones sobre un string se realizan completas (no se interrumpen a mitad).

- SET: guarda un valor en una clave. SET nombre "Juana"
- GET: obtiene el valor de la clave. GET nombre --> "Juana"
- DEL: elimina la clave. DEL nombre

Si el string contiene un número (entero o flotante), Redis lo trata como tal

Strings

Si el string contiene un número (entero o flotante), Redis lo trata como tal

- INCR / DECR: incrementa o decrementa en 1.

SET contador 10

INCR contador → 11

DECR contador → 10

- INCRBY / DECRBY: incrementa/decrementa en un valor específico.

INCRBY contador 5 → 15

- INCRBYFLOAT: incrementa con números decimales.

INCRBYFLOAT contador 2.5

Strings

Redis permite manipular los strings como si fueran arrays

- APPEND: agregar texto al final.

APPEND nombre "Perez"

GET nombre → "Juana Perez"

- GETRANGE: obtener un rango de caracteres.

GETRANGE nombre 0 4 → "Juana"

- SETRANGE: reemplazar parte de la cadena.

Strings

Redis permite manipular los strings como si fueran arrays

- APPEND: agregar texto al final.

APPEND nombre "Perez"

GET nombre → "Juana Perez"

- GETRANGE: obtener un rango de caracteres.

GETRANGE nombre 0 4 → "Juana"

- SETRANGE: reemplazar parte de la cadena.

Listas

Secuencia ordenada de strings. Pueden usarse como colas (FIFO) o pilas (LIFO).

- LPUSH tareas "hacer café" # Agregar al inicio
- RPUSH tareas "leer mails" # Agregar al final
- LRANGE tareas 0 -1 # Ver todos los elementos con -1
- LPOP tareas # Saca el primer elemento
- RPOP tareas # Saca el último
- LLEN tareas # Cantidad de elementos

Hashes

Estructura tipo diccionario (clave → campo:valor).

Ideal para representar objetos (ej: usuarios).

- HSET usuario:1 nombre "Ana" edad 25 ciudad "La Plata"
- HGET usuario:1 nombre → "Ana"
- HGETALL usuario:1 # Devuelve todo el hash
- HDEL usuario:1 edad # Elimina un campo
- HLEN usuario:1 # Cantidad de campos
- HEXISTS usuario:1 ciudad → 1 (true)

Un hash es como un diccionario o un objeto JSON:

1. La clave principal (ejemplo: usuario:1) identifica al hash.
2. Dentro del hash hay campos y valores (ejemplo: nombre → Ana, edad → 25).

Sets (Conjuntos)

Colección de elementos únicos, sin orden.

- SADD amigos:ana "juan" "pedro" "lucia"
- SMEMBERS amigos:ana # Lista de amigos
- SISMEMBER amigos:ana "juan" → 1 #Devuelve 1 si sí está, 0 sino
- SREM amigos:ana "pedro" #elimina pedro
- SCARD amigos:ana # Cantidad de elementos
- SINTER amigos:ana amigos:juan # Amigos en común
- SUNION amigos:ana amigos:maria # Todos los amigos
- SDIFF amigos:ana amigos:maria # Diferencia
- KEYS amigos:* #Sets que empiezan con amigos:* ej:
"amigos:ana", "amigos:juan", "amigos:maria"

Sorted Sets (Zsets)

Conjunto ordenado por puntaje numérico.

Muy usado en rankings y puntuaciones.

- ZADD ranking 100 "Ana" 120 "Luis" 95 "Carlos"
- ZRANGE ranking 0 -1 WITHSCORES # Ascendente
- ZREVRANGE ranking 0 2 WITHSCORES # Top 3 descendente
- ZSCORE ranking "Luis" → 120
- ZINCRBY ranking 10 "Ana" # Aumenta el score
- ZCARD ranking # Cantidad de elementos

Bitmaps

Permiten manipular bits individuales en un string.

Muy útiles para marcar presencia/ausencia.

- SETBIT asistencia 1 1 # Usuario 1 estuvo presente
- SETBIT asistencia 2 0 # Usuario 2 ausente
- GETBIT asistencia 1 → 1
- BITCOUNT asistencia # Cantidad de presentes
- BITOP AND resultado asistencia1 asistencia2 # Comparar

Geoespacial

Almacena coordenadas (lat, lon) y permite consultas geográficas.

- GEOADD lugares 13.361389 38.115556 "Palermo"
- GEOADD lugares 15.087269 37.502669 "Catania"
- GEODIST lugares Palermo Catania km → 166.27
- GEORADIUS lugares 15 37 200 km # Dentro de 200 km
- GEOPOS lugares Palermo # Lat/Lon